

NGD-SLAM: Towards Real-Time Dynamic SLAM without GPU

Yuhao Zhang^{1,2}, Mihai Bujanca³, Mikel Luján¹

Abstract—Many existing visual SLAM methods can achieve high localization accuracy in dynamic environments by leveraging deep learning to mask moving objects. However, these methods incur significant computational overhead as the camera tracking needs to wait for the deep neural network to generate mask at each frame, and they typically require GPUs for real-time operation, which restricts their practicality in real-world robotic applications. Therefore, this paper proposes a real-time dynamic SLAM system that runs exclusively on a CPU. Our approach incorporates a mask propagation mechanism that decouples camera tracking and deep learning-based masking for each frame. We also introduce a hybrid tracking strategy that integrates ORB features with optical flow methods, enhancing both robustness and efficiency by selectively allocating computational resources to input frames. Compared to previous methods, our system maintains high localization accuracy in dynamic environments while achieving a tracking frame rate of 60 FPS on a laptop CPU. These results demonstrate the feasibility of utilizing deep learning for dynamic SLAM without GPU support. Since most existing dynamic SLAM systems are not open-source, we make our code publicly available at: <https://github.com/yuhaozhang7/NGD-SLAM>

I. INTRODUCTION

Simultaneous Localization and Mapping (SLAM) is a set of algorithms that leverage sensors' input data to achieve accurate, centimeter-level localization and to build a map of the surroundings simultaneously. It is a foundation for applications like robotics and augmented reality.

One focus in SLAM research is visual SLAM, which refers to using cameras for localization and mapping. Although numerous visual SLAM algorithms [1, 2, 3, 4] have demonstrated high tracking accuracy across various scenarios, most of them are designed under the assumption of a static environment. These methods typically encounter notable performance degradation in dynamic settings [5].

To address this challenge, we propose NGD-SLAM (No GPU Dynamic SLAM), a real-time visual SLAM system for dynamic environments that operates entirely on a CPU.

Why CPU Only? In visual dynamic SLAM, a common approach is to incorporate deep learning models to mask dynamic objects in input images using semantic information. These methods demonstrate significant improvements compared to approaches that filter dynamic features purely based on geometric constraints [6]. However, these models introduce high computational costs, requiring a GPU to achieve real-time performance on most modern cameras (30 Hz), making them impractical for embedded systems, where weight, power, and heat constraints are critical considerations or GPU may not be available.

Why Real-Time? One important property of SLAM is its ability to operate online and provide instantaneous pose estimation and map building, both of which are critical for many robotic applications. If a SLAM system operates at a lower frame rate than the sensor rate, it can result in accumulated latency, causing outdated pose estimates; otherwise, it has to drop some input frames to match the system's frame rate, which may degrade performance in high-motion settings due to increased spatial and temporal discrepancies between adjacent frames. However, many existing dynamic SLAM methods have spent too much computational power on improving tracking accuracy while overlooking the importance of real-time performance [7, 8].

Therefore, there has been an urgent need to develop robust and efficient dynamic SLAM for low-power devices. Our system is built upon ORB-SLAM3 framework [2] and includes an additional semantic thread for dynamic object detection using YOLO [9] like many existing methods. The core contribution of this paper is the tracking pipeline shown in Figure 1. Rather than using deep learning model less frequently during the tracking process [10, 11, 12], we approach the challenge of improving efficiency by associating previous results with the current frame. Specifically, we introduce a straightforward yet highly effective **mask propagation mechanism** (Figure 2) that leverages previous segmentation results from semantic thread to predict the current dynamic mask within a few milliseconds. This mechanism enables fully parallel camera tracking and dynamic object segmentation. Considering that frames are not equally important [13, 14], we also employ a **hybrid tracking strategy** that combines the advantages of optical flow and ORB feature tracking to enable better usage of computational resources for each frame and provides much more efficient and robust tracking in dynamic environments.

Through experiments on public datasets, our system demonstrates localization accuracy comparable to that of SOTA methods. When taking real-time performance into consideration, it shows a clear advantage over existing methods, with a tracking frame rate of 60 FPS on a laptop CPU.

The main contributions of this paper are:

- An RGB-D SLAM system that not only achieves real-time performance without a GPU but also maintains localization accuracy comparable to state-of-the-art (non-real-time) methods in dynamic environments.
- A mask propagation mechanism that efficiently estimates current dynamic object masks based on previous masks, mitigating the high latency caused by the deep learning model.
- A hybrid usage of optical flow and ORB features for

¹University of Manchester. ²University of Cambridge.

³Qualcomm Technologies XR Labs, Austria.

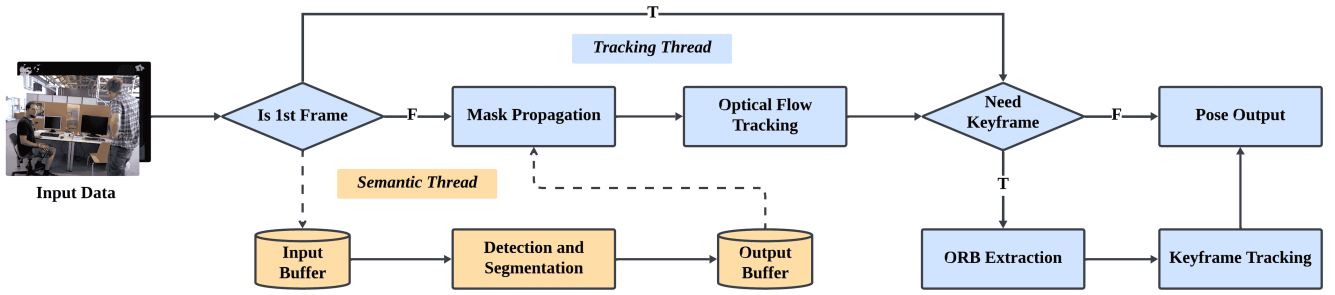


Fig. 1: Tracking Pipeline of NGD-SLAM. Based on a mask propagation mechanism, a pixel-wise mask is generated from previous mask to exclude dynamic features. Optical flow is employed to efficiently track the remaining static features across non-keyframes, while ORB feature matching is maintained to establish data association among keyframes.

camera tracking that spend less computation on less important frames (non-keyframes), while maintaining robustness in dynamic environments.

II. RELATED WORK

To address the challenges of dynamic environments, it is crucial to identify pixels or keypoints originating from dynamic objects. With advancements in deep learning, several methods have been developed for detecting or segmenting dynamic instances across input images using semantic information. Based on ORB-SLAM2 [1], DS-SLAM [15] improves localization accuracy in dynamic environments by incorporating SegNet [16] for dynamic instances segmentation while maintaining epipolar geometry constraints for moving consistency check. Similarly, DynaSLAM [17] employs Mask R-CNN [18] and multi-view geometry to eliminate dynamic ORB features. TRS-SLAM [11] steps further towards efficiency by introducing a geometry module that handles unknown moving objects and applying a segmentation model only to keyframes. More recently, CFP-SLAM [19] achieves high tracking accuracy by proposing a coarse-to-fine static probability approach to enhance localization accuracy by calculating and updating static probabilities as weights for pose optimization. Leveraging the advancements in large segmentation models, USD-SLAM [20] utilizes SAM [21] and SegGPT [22] to identify movable objects and applies motion state constraints to handle dynamic instances.

Beyond feature-based methods, some works focus on dense 3D reconstruction and camera tracking in dynamic environments. ReFusion [23] uses direct tracking on the TSDF and GPU parallelization to detect dynamics through residuals from pose estimation, while ACEFusion [24] integrates semantic information and dense optical flow to segment dynamic objects and perform both static and dynamic reconstruction on a limited power budget.

Although incorporating a segmentation model for tracking in dynamic environments gives remarkable improvements in accuracy, algorithms adopting this method typically rely on powerful GPUs to achieve (near) real-time performance due to the computationally intensive nature of deep neural networks. One strategy to improve efficiency is to incorporate a separate semantic thread for the deep learning model. Based on this idea, several approaches have been proposed to parallelize feature extraction and dynamic object

identification [15, 19]. However, these methods offer only limited gains in efficiency due to the inherent dependency between the tracking process and the deep neural network. Alternatively, some algorithms apply deep learning only to keyframes [10, 11, 12], which significantly reduces the average tracking time but compromises accuracy.

While NGD-SLAM also includes a separate semantic thread for the deep neural network, it incorporates a mask propagation mechanism that allows the tracking thread to operate independently from the semantic thread. Moreover, distinct from the methods that apply dense optical flow or scene flow for differentiating dynamic points from static ones [15, 24, 25, 26, 27, 28], NGD-SLAM leverages optical flow to track sparse dynamic and static features, enhancing the system's accuracy and efficiency.

III. METHODOLOGY

In the paper, we refer FPS over 30 as real-time, which is the sensor rate of TUM [29] and BONN [23] datasets.

A. Tracking Pipeline

Figure 1 illustrates the tracking pipeline of NGD-SLAM. Initially, the image data of the first frame is forwarded to the semantic thread (Section III-B), and this frame is set as a keyframe. ORB features are then extracted and map is initialized as part of the keyframe tracking (Section III-D.2). For subsequent frames, the system employs mask propagation mechanism (Section III-C) to generate dynamic masks based on previous masks, and uses optical flow to track static keypoints from the last frame and estimates the camera's pose (Section III-D.1). The decision to set a new keyframe is then made. If a new keyframe is required, the system extracts new ORB features from the current frame and discard the features within the mask, and continues with the keyframe tracking using ORB features (Section III-D.2).

B. Semantic Thread

NGD-SLAM achieves better run-time performance by adding a semantic thread that detects dynamic objects using the YOLO model [9]. This thread works with an input and output buffer, each holding one raw image and the corresponding depth (input) or mask (output). Frames that pass from the tracking thread to the input buffer are processed by the network, then moved to the output buffer to be

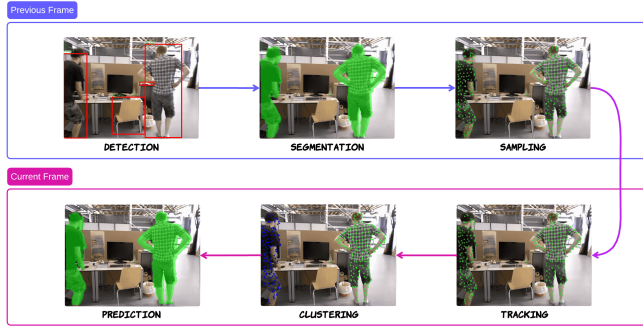


Fig. 2: Mask Propagation Pipeline. The detection and segmentation are processed within the semantic thread, while the tracking thread utilizes these results to predict the current mask (the current frame and the previous frame may appear visually similar but are actually different).

accessed by the tracking thread. This setup ensures that the semantic thread processes the most recent input frame and the tracking thread does not have to wait for the result from the semantic thread. Instead, it directly takes the mask and corresponding image in the output buffer (without placement) and uses them to predict the current mask. Although there is a temporal mismatch because two threads get invoked at different frequencies, we compensate for this by the mask propagation.

C. Mask Propagation

One important module of NGD-SLAM is the mask propagation mechanism. Compared to the forward process of a deep neural network, the mask propagation is much faster, taking only a few milliseconds to process one frame on a CPU. It can be expressed as:

$$M_i = f_{\text{prop}}(I_i, I_{i-n}, D_{i-n}, M_{i-n}), \quad n > 0,$$

where I_{i-n} , D_{i-n} , M_{i-n} represent the raw image, depth image, and mask of the $(i - n)^{\text{th}}$ frame, respectively. A smaller n will result in a faster f_{prop} .

While mask propagation is also widely studied in video segmentation using deep learning [30], we adopt the classical image processing techniques for NGD-SLAM, as our goal is to achieve supreme efficiency without a GPU, specifically for SLAM applications. Figure 2 shows the pipeline of the mask propagation, which consists of six components.

1) *Detection*: We employ a relative lightweight YOLO model for the semantic thread, which allows the temporal difference between I_i and I_{i-n} remains small (Figure 2).

2) *Segmentation*: Using an object detection model is more efficient than a segmentation network but it only provides bounding boxes for detected objects. To improve this, we use depth information to segment the objects within these boxes, assuming the detection model can accurately locate the object and provide a tight bounding box, such that the detected object occupies most of the box. Subsequently, the connected component labeling algorithm [31] is used to exclude masks of objects that share a similar depth but are not part of the main object, and a dilation process further refines the mask before it is passed into the output buffer.

3) *Sampling*: Given the masks of identified dynamic objects, dynamic points are extracted within the mask. The image is divided into cells with size of 15×15 . Within each cell, pixels that are masked and meet the FAST keypoint threshold [2] are identified, and the one with the highest value is selected. If there is no suitable FAST keypoint, a random pixel is chosen within that cell. This grid structure allows for uniform extraction of dynamic keypoints, and the tracking stability is ensured due to the distinctiveness of keypoints and their association with high-gradient regions.

4) *Tracking*: The system utilizes the Lucas-Kanade optical flow method to track sampled dynamic keypoints, which assumes that the grayscale value of a keypoint remains constant across different frames [14, 32]. Mathematically, this is represented as:

$$I(x + dx, y + dy, t + dt) = I(x, y, t)$$

where $I(x, y, t)$ is the grayscale intensity value of a pixel at coordinates (x, y) with time t . This equation can be further expanded as:

$$I(x + dx, y + dy, t + dt) \approx I(x, y, t) + \frac{\partial I}{\partial x} \cdot dx + \frac{\partial I}{\partial y} \cdot dy + \frac{\partial I}{\partial t} \cdot dt$$

Subsequently, assuming that pixels move uniformly within a small window, an optimization problem can be formulated using the pixel gradients within the window to estimate pixel motion (dx, dy) by minimizing the total grayscale error of matched pixels. For large n , tracking will be performed across the intermediate frames between I_i and I_{i-n} .

5) *Clustering*: For tracked dynamic keypoints in the current frame, the DBSCAN algorithm [33] is applied to cluster points that are close to each other while labeling those significantly distant from any cluster as noise. Employing this method allows for the effective distinction between tracked keypoints originating from distinct dynamic objects and filters out the outliers.

6) *Prediction*: After deriving the clusters, the system becomes aware of the dynamic entity's location within the current frame. It can then use the depth information of points in each cluster to obtain the corresponding mask with precise shape of the dynamic object, as shown in Figure 2.

In cases where the segmentation model fails to provide a mask, such as when YOLO struggles to detect objects due to camera rotation, the system uses the mask from the last frame to predict the current mask:

$$M_i^p = \begin{cases} f_{\text{prop}}(I_i, I_{i-1}, D_{i-1}, M_{i-1}^p), & \text{if buffer empty,} \\ f_{\text{prop}}(I_i, I_{i-n}, D_{i-n}, M_{i-n}^s), & \text{otherwise.} \end{cases}$$

The superscripts denote whether the mask is predicted using the mask propagation mechanism (p) or generated by the segmentation model (s). This method allows for continuous tracking of dynamic keypoints until a dynamic object is no longer visible to the camera, even if a deep neural network fails to detect objects. By continually sampling new points from previous mask, rather than tracking the same points across multiple frames, it ensures that dynamic points are up-to-date and enhances tracking robustness (Figure 3).

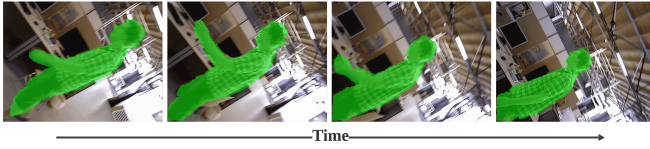


Fig. 3: Continuous Mask Propagation. The system continuously tracks a dynamic person over 60 frames by keep propagating the mask from the last frame (M_{i-1}^p), when YOLO completely fails with the rotated images.

D. Hybrid Strategy

1) *Optical Flow Tracking*: In contrast to ORB-SLAM3, which depends on ORB feature matching for every input frame, NGD-SLAM uses the LK optical flow method [32] to track the static keypoints from the last frame. Geometric constraints are formed by associating 3D map points from the last frame to tracked 2D keypoints in the current frame. The pose is then estimated using the EPnP [34], combined with RANSAC [35] in an iterative manner.

This design is based on two considerations. Firstly, the mask propagation mechanism typically fails when a dynamic object is presented in the current frame but doesn't appear in the frame used for prediction. Thus, extracting new ORB features for each frame and matching with local map points based on the differences in descriptor values can result in mismatches (Figure 4). In contrast, optical flow tracking, which only tracks the static keypoints from the last frame based on image gradients and filters out outliers for future tracking, allows for the tracking of the same static points across multiple frames and tends to be more robust in this case. Secondly, the optical flow tracking skips the computationally expensive ORB feature extraction for non-keyframes, making the tracking process much more efficient.

2) *ORB Keyframe Tracking*: ORB features are retained for keyframes to establish connections between them, allowing the system to maintain essential ORB-SLAM3 components such as co-visible graphs and local maps, which are crucial for robust tracking and efficient data retrieval. Keyframes are tracked by first using the pose T_t^{init} to project map points into the current frame to get matches, where T_t^{init} is defined as:

$$T_t^{\text{init}} = \begin{cases} T_t^{\text{flow}}, & \text{if optical flow succeeds,} \\ (T_{t-1}T_{t-2}^{-1})T_{t-1}, & \text{otherwise.} \end{cases}$$

Here, T_t^{flow} is the pose of the current frame estimated using optical flow tracking. To get the accurate pose, optimization is performed by minimizing the reprojection error:

$$e = \frac{1}{2} \sum_{i=1}^n \left\| u_i - \frac{1}{s_i} K T P_i \right\|_2^2$$

using the Levenberg-Marquardt method. This optimization process is initialized with T_t^{init} to facilitate faster convergence. Such keyframe tracking also serves as a backup tracking method when optical flow tracking fails, as ORB features are more robust to illumination changes and motion blur. Similar to ORB-SLAM3 [2], new map points are added after tracking, and the keyframe is passed to the local mapping thread for further optimization.



Fig. 4: Comparison of ORB Feature Tracking and Optical Flow Tracking. (Top) Extracting new ORB features for each frame and performing matching with map points lead to matches of dynamic points, as shown in the red box. (Bottom) Optical flow tracks same static points across multiple frames, which works well even when mask propagation underperforms due to the newly appeared dynamic person not being detected in previous frames.

Compared with many SLAM algorithms that do not distinguish between the computation allocated to keyframes and non-keyframes in frontend tracking, our hybrid usage of ORB features and optical flow to track keypoints allows our system to spend significantly fewer computation on non-keyframes tracking, thereby improving efficiency.

IV. EXPERIMENTS

A. Experimental Setup

Several state-of-the-art and classic feature-based dynamic SLAM algorithms are chosen for comparison, including DynaSLAM [17], DS-SLAM [15], RDS-SLAM [10], TRS-SLAM [11], CFP-SLAM [19], USD-SLAM [20], and DFS-SLAM [36], as well as dense SLAM methods like ReFusion [23] and ACEFusion [24].

All highly dynamic sequences from the TUM dataset [29] are utilized, which captures scenarios of two individuals moving around a desk, one wearing a plaid shirt that is rich in texture. We also evaluate the algorithms on sequences of BONN datasets [23], which include situations such as many people walking randomly, a person moving a static object in the scene, several people moving side by side, and the camera continuously tracking a walking person. Two datasets are both captured with a camera operating at 30 Hz.

For metrics, ATE (Absolute Trajectory Error) and RPE (Relative Pose Error) [37] are chosen as evaluation metrics, with the interval for RPE measurements set to 1 second (30 frames in the case of TUM and BONN). Tracking time per frame is used to measure the efficiency in milliseconds. As some algorithms are not open-sourced and are tested on various devices, a relative fair comparison of efficiency is ensured by displaying a ratio defined as:

$$\text{Ratio} = \frac{\text{Tracking time of the system}}{\text{Tracking time of ORB-SLAM2}},$$

where the information is obtained from their source paper (ORB-SLAM2 is a strong SLAM system for static environments and serves as a common base framework).

All experiments are conducted on a laptop equipped with an AMD Ryzen 7 4800H CPU and an NVIDIA GeForce RTX 2060 GPU, and GPU is not used for our method. The presented results are obtained by processing all frames in the sequence, even for methods that are not real-time.

TABLE I: Comparison of RMSE for ATE (m), RPE translation (m/s), and RPE rotation ($^{\circ}$ /s) on TUM Highly Dynamic Sequences.

	DynaSLAM			DS-SLAM			RDS-SLAM			TRS-SLAM*		
	ATE (m)	RPE (m/s)	RPE ($^{\circ}$ /s)	ATE (m)	RPE (m/s)	RPE ($^{\circ}$ /s)	ATE (m)	RPE (m/s)	RPE ($^{\circ}$ /s)	ATE (m)	RPE (m/s)	RPE ($^{\circ}$ /s)
f3/w_xyz	0.015	0.021	0.452	0.025	0.033	0.826	0.057	0.042	0.922	0.019	0.023	0.636
f3/w_rpy	0.036	0.045	0.902	0.444	0.150	3.004	0.160	0.132	13.169	0.037	0.047	1.058
f3/w_half	0.027	0.028	0.737	0.030	0.030	0.814	0.081	0.048	1.883	0.029	0.042	0.965
f3/w_static	0.007	0.008	0.258	0.008	0.010	0.269	0.008	0.022	0.494	0.011	0.011	0.287
	CFP-SLAM*			USD-SLAM			DFS-SLAM			Ours*		
	ATE (m)	RPE (m/s)	RPE ($^{\circ}$ /s)	ATE (m)	RPE (m/s)	RPE ($^{\circ}$ /s)	ATE (m)	RPE (m/s)	RPE ($^{\circ}$ /s)	ATE (m)	RPE (m/s)	RPE ($^{\circ}$ /s)
f3/w_xyz	0.015	0.019	0.620	0.035	–	–	0.013	–	–	0.015	0.020	0.470
f3/w_rpy	0.041	0.054	1.052	0.035	–	–	0.027	–	–	0.034	0.044	0.889
f3/w_half	0.023	0.027	0.785	0.020	–	–	0.018	–	–	0.024	0.025	0.695
f3/w_static	0.007	0.009	0.254	–	–	–	0.006	–	–	0.007	0.009	0.262

* indicates methods that are real-time or near real-time, **red** indicates the best among all algorithms, and **blue** indicates the best among all (near) real-time algorithms.

TABLE II: Comparison of RMSE for ATE (m) on BONN Dataset.

	DynaSLAM	ReFusion	ACEFusion*	USD	Ours*
crowd	0.016	0.204	0.016	–	0.024
crowd2	0.031	0.155	0.027	0.028	0.025
crowd3	0.038	0.137	0.023	0.026	0.033
mov_no_box	0.232	0.071	0.070	–	0.016
mov_no_box2	0.039	0.179	0.029	0.178	0.036
person_track	0.061	0.289	0.070	0.028	0.046
person_track2	0.078	0.463	0.071	0.041	0.062
synchronous	0.015	0.441	0.014	–	0.028
synchronous2	0.009	0.022	0.010	–	0.009

B. Accuracy Analysis

Table I presents a comparison of our system against various baselines using the TUM dataset, with the minimum error values highlighted. DFS consistently outperforms all four sequences due to its extensive extra computations for handling dynamic objects and static points optimization [36]. Our method achieves similar performance with slightly higher error, but when real-time performance is considered, it achieves the highest accuracy overall. While CFP also performs well in near real-time, it is less accurate in the *f3/w_rpy* sequence as the epipolar constraints it employs are less effective in scenarios involving rotation [19].

Table II shows the results on the BONN dataset. Only Dyna and USD from previous experiment can be used for comparison, as other methods are either not evaluated on this dataset or are not open-sourced, so ReFusion and ACEFusion are added for comparison. Our system outperforms other methods across several sequences. It also has certain ability to handle situations where a walking person is carrying a static object, benefiting from the depth segmentation and the connected component algorithm. This shows the robustness of the system across a wider range of dynamic scenes.

C. Runtime Analysis

Table III and Figure 5 presents a comparison of the tracking time ratio across TUM sequences. The average tracking time of ORB-SLAM2 is about 20 ms on our experimental device; therefore, a value below 1.65 can be considered real-time (30 FPS in this paper). While DFS and USD achieve high accuracy, both fall significantly short of real-time performance. Notably, DFS requires more than $3\times$ the processing time of its base framework ORB-SLAM2, even

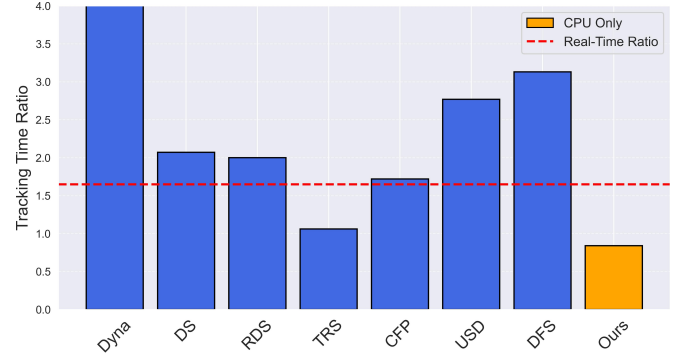


Fig. 5: Comparison of the tracking time ratio across different methods.

with GPU. TRS demonstrates clear real-time performance with minimal additional computation for dynamic environments; however, it compromises accuracy by applying deep learning models only to keyframes. In contrast, our system achieves real-time performance with only a CPU while maintaining accuracy comparable to state-of-the-art method. Although some algorithms discussed in the previous section [23, 24] cannot be fairly compared here, none of them can achieve (near) real-time performance without GPU.

D. Real-Time Analysis

The average processing time per frame for the system is 16.72 ms (≈ 60 FPS), attributed to the efficiency of optical flow tracking in non-keyframes. Since only the tracking of a keyframe involves all components of NGD-SLAM, we also evaluate the average time required for tracking a keyframe using mask propagation, optical flow tracking, and ORB feature tracking, as shown in Table IV. It is evident that optical flow tracking is highly efficient, and although mask propagation requires slightly more time, the combined tracking process falls within 33.3 ms (30 FPS), which still meets the criteria for real-time performance for the TUM and BONN datasets. Notably, while YOLO detection takes approximately 30 to 40 milliseconds per frame, it is not factored into this tracking time since the tracking and semantic threads operate concurrently at different frequencies.

E. Ablation Study

Table V presents an ablation study analyzing the impact of different system components. The base framework, ORB-SLAM3 [2], fails to localize in dynamic environments,

TABLE III: Comparison of Tracking Time Ratio (ORB-SLAM2/3 are for Static Environment).

	ORB-SLAM2	ORB-SLAM3	DynaSLAM	DS-SLAM	RDS-SLAM	TRS-SLAM	CFP-SLAM	USD-SLAM	DFS-SLAM	NGD-SLAM
Ratio	1.00	0.98	15.94	2.07	2.00	1.06	1.72	2.77	3.13	0.84
Real-Time	✓	✓	✗	✗	✗	✓	✓ (near)	✗	✗	✓
Device	CPU	CPU	CPU + GPU	CPU + GPU	CPU + GPU	CPU + GPU	CPU + GPU	CPU + GPU	CPU + GPU	CPU

TABLE IV: Average Tracking Time per Keyframe (ms).

MP	OF Tracking	ORB Tracking	Total
7.92	3.04	19.73	30.69

TABLE V: ATE RMSE (m) of Different Configurations.

	ORB3	+YOLO	+MP	+MP+HS	Improve
f3/w_xyz	0.390	0.014	0.014	0.015	96%
f3/w_rpy	0.639	0.041	0.037	0.034	95%
f3/w_half	0.431	0.023	0.028	0.024	94%
f3/w_static	0.096	0.007	0.007	0.007	93%
Time / Frame	20 ms	54 ms	28 ms	17 ms	15%

MP (Mask Propagation), HS (Hybrid Tracking Strategy).

exhibiting consistently high errors across all sequences. Integrating a deep learning-based mask significantly reduces errors by filtering out unreliable dynamic keypoints. However, it introduces substantial computational overhead, making real-time tracking infeasible. Mask propagation addresses this by using only previously generated masks rather than waiting for the deep learning model to compute new mask for current frame, enabling real-time tracking on a CPU. Finally, our full system, incorporating a hybrid tracking strategy, significantly reducing tracking time. By applying fast optical flow tracking rather than ORB feature tracking to non-keyframes, it minimizes the computation allocated to less critical frames.

F. The Importance of Real-Time

Most existing dynamic SLAM systems use all frames from the datasets during evaluation, even when systems themselves are not real-time. However, such evaluation is insufficient for practical SLAM applications. If a system operates at a lower frame rate than the sensor, latency can accumulate. To prevent latency accumulation, which is critical for most SLAM applications that requires instantaneous pose estimation, systems must selectively drop frames, thus matching the input frame rate to the system frame rate. This consideration is frequently overlooked in evaluations of many recent dynamic SLAM algorithms.

Since most algorithms are not publicly available, we use DynaSLAM as a reference and evaluated on TUM sequences with reduced input frame rates. As shown in Figure 6, the impact of reduced FPS is minor for sequences involving moderate camera movements. However, in the *f3/w_rpy* sequence, characterized by rapid camera motion, errors significantly increase and tracking failures occur due to larger temporal gaps between frames at lower frame rates.

Although we were unable to experimentally evaluate SOTA methods such as DFS-SLAM under conditions of dropped frames due to their closed-source nature, it is very likely that similar performance degradation would also occur in non-real-time methods other than DynaSLAM.

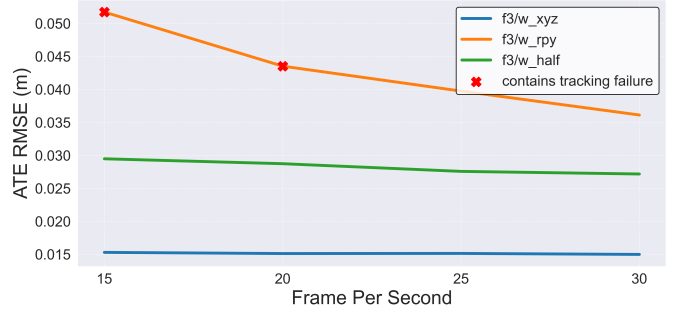


Fig. 6: DynaSLAM on TUM Sequences with Reduced Input Frame Rate.

V. CONCLUSION

This paper presents a real-time visual SLAM system for dynamic environments that runs on a CPU while maintaining high tracking accuracy. We improve efficiency by decoupling tracking thread from semantic thread at each time step using mask propagation, which can also be easily adapted to other SLAM systems given its generic and modularized design. While this mechanism is based on classical image processing techniques (as we aim to show such a simple method can be highly effective), the broader concept of mask propagation would allow dynamic SLAM systems to leverage deep learning-based mask propagation for better generalization and robustness, depending on the available resources. In addition, we also propose a hybrid tracking strategy that prioritizes critical frames, reducing computational cost at frontend. Evaluations show that our method achieves accuracy close to SOTA while outperforming real-time methods. Given its high efficiency, we believe it can further benefit from higher-frame-rate cameras, enhancing localization accuracy and high-frequency pose estimation.

VI. LIMITATION AND FUTURE WORK

One limitation of our system is that the mask propagation relies on RGB-D inputs for depth information. To adapt it for monocular or stereo SLAM, one could simply use a segmentation model directly rather than a detection model or integrate a learning-based depth estimation model.

Another limitation is that we do not explicitly handle dynamic instances that do not belong to the classes defined in the detection model or static objects that are movable. However, given the modular nature of mask propagation, it is easy to leverage recent advancements in open-vocabulary segmentation and universal segmentation with motion constraints to identify moving objects, like USD-SLAM [20].

ACKNOWLEDGEMENT

Huge thanks to Ruiqi Ye (University of Manchester) for the insightful discussions and valuable comments on paper.

REFERENCES

- [1] R. Mur-Artal and J. D. Tardós, “Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras,” *IEEE transactions on robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [2] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. Montiel, and J. D. Tardós, “Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [3] T. Qin, P. Li, and S. Shen, “Vins-mono: A robust and versatile monocular visual-inertial state estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [4] C. Forster, M. Pizzoli, and D. Scaramuzza, “Svo: Fast semi-direct monocular visual odometry,” in *2014 IEEE international conference on robotics and automation (ICRA)*, pp. 15–22, IEEE, 2014.
- [5] N. Radulov, Y. Zhang, M. Bujanca, R. Ye, and M. Luján, “A framework for reproducible benchmarking and performance diagnosis of slam systems,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 14225–14232, IEEE, 2024.
- [6] M. R. U. Saputra, A. Markham, and N. Trigoni, “Visual slam and structure from motion in dynamic environments: A survey,” *ACM Computing Surveys (CSUR)*, vol. 51, no. 2, pp. 1–36, 2018.
- [7] C. Galagain, M. Poreba, and F. Goulette, “Is semantic slam ready for embedded systems? a comparative survey,” *arXiv preprint arXiv:2505.12384*, 2025.
- [8] Y. Wang, Y. Tian, J. Chen, K. Xu, and X. Ding, “A survey of visual slam in dynamic environment: The evolution from geometric to semantic approaches,” *IEEE Transactions on Instrumentation and Measurement*, 2024.
- [9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 779–788, 2016.
- [10] Y. Liu and J. Miura, “Rds-slam: Real-time dynamic slam using semantic segmentation methods,” *Ieee Access*, vol. 9, pp. 23772–23785, 2021.
- [11] T. Ji, C. Wang, and L. Xie, “Towards real-time semantic rgb-d slam in dynamic environments,” in *2021 IEEE international conference on robotics and automation (ICRA)*, pp. 11175–11181, IEEE, 2021.
- [12] F. Zhong, S. Wang, Z. Zhang, and Y. Wang, “Detect-slam: Making object detection and slam mutually beneficial,” in *2018 IEEE winter conference on applications of computer vision (WACV)*, pp. 1001–1010, IEEE, 2018.
- [13] S. Leutenegger, S. Lynen, M. Bosse, R. Siegwart, and P. Furgale, “Keyframe-based visual-inertial odometry using nonlinear optimization,” *The International Journal of Robotics Research*, vol. 34, no. 3, pp. 314–334, 2015.
- [14] X. Gao, T. Zhang, Y. Liu, and Q. Yan, *14 Lectures on Visual SLAM: From Theory to Practice*. Publishing House of Electronics Industry, 2017.
- [15] C. Yu, Z. Liu, X.-J. Liu, F. Xie, Y. Yang, Q. Wei, and Q. Fei, “Ds-slam: A semantic visual slam towards dynamic environments,” in *2018 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pp. 1168–1174, IEEE, 2018.
- [16] V. Badrinarayanan, A. Kendall, and R. Cipolla, “Segnet: A deep convolutional encoder-decoder architecture for image segmentation,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 39, no. 12, pp. 2481–2495, 2017.
- [17] B. Bescos, J. M. Fàcil, J. Civera, and J. Neira, “Dynaslam: Tracking, mapping, and inpainting in dynamic scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [18] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask r-cnn,” in *Proceedings of the IEEE international conference on computer vision*, pp. 2961–2969, 2017.
- [19] X. Hu, Y. Zhang, Z. Cao, R. Ma, Y. Wu, Z. Deng, and W. Sun, “Cfp-slam: A real-time visual slam based on coarse-to-fine probability in dynamic environments,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4399–4406, IEEE, 2022.
- [20] J. Wang, Y. Ren, Z. Li, X. Xie, Z. Chen, T. Shen, H. Liu, and K. Wang, “Usd-slam: A universal visual slam based on large segmentation model in dynamic environments,” *IEEE Robotics and Automation Letters*, 2024.
- [21] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, et al., “Segment anything,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 4015–4026, 2023.
- [22] X. Wang, X. Zhang, Y. Cao, W. Wang, C. Shen, and T. Huang, “Seggpt: Segmenting everything in context,” *arXiv preprint arXiv:2304.03284*, 2023.
- [23] E. Palazzolo, J. Behley, P. Lottes, P. Giguere, and C. Stachniss, “Refusion: 3d reconstruction in dynamic environments for rgb-d cameras exploiting residuals,” in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 7855–7862, IEEE, 2019.
- [24] M. Bujanca, B. Lennox, and M. Luján, “Acefusion-accelerated and energy-efficient semantic 3d reconstruction of dynamic scenes,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 11063–11070, IEEE, 2022.
- [25] T. Zhang, H. Zhang, Y. Li, Y. Nakamura, and L. Zhang, “Flowfusion: Dynamic dense rgb-d slam based on optical flow,” in *2020 IEEE international conference on robotics and automation (ICRA)*, pp. 7322–7328, IEEE, 2020.
- [26] M. Bujanca, M. Luján, and B. Lennox, “Fullfusion: A framework for semantic reconstruction of dynamic scenes,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*, pp. 0–0, 2019.
- [27] H. Jiang, Y. Xu, K. Li, J. Feng, and L. Zhang, “Rodyn-slam: Robust dynamic dense rgb-d slam with neural radiance fields,” *IEEE Robotics and Automation Letters*, 2024.
- [28] Y. Wang and S. Huang, “Towards dense moving object segmentation based robust dense rgb-d slam in dynamic scenarios,” in *2014 13th International Conference on Control Automation Robotics & Vision (ICARCV)*, pp. 1841–1846, IEEE, 2014.
- [29] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, “A benchmark for the evaluation of rgb-d slam systems,” in *2012 IEEE/RSJ international conference on intelligent robots and systems*, pp. 573–580, IEEE, 2012.
- [30] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, and C. Feichtenhofer, “Sam 2: Segment anything in images and videos,” *arXiv preprint arXiv:2408.00714*, 2024.
- [31] L. Di Stefano and A. Bulgarelli, “A simple and efficient connected components labeling algorithm,” in *Proceedings 10th international conference on image analysis and processing*, pp. 322–327, IEEE, 1999.
- [32] B. D. Lucas and T. Kanade, “An iterative image registration technique with an application to stereo vision,” in *IJCAI’81: 7th international joint conference on Artificial intelligence*, vol. 2, pp. 674–679, 1981.
- [33] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, et al., “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *kdd*, vol. 96, pp. 226–231, 1996.
- [34] V. Lepetit, F. Moreno-Noguer, and P. Fua, “Ep n p: An accurate o(n) solution to the p n p problem,” *International journal of computer vision*, vol. 81, pp. 155–166, 2009.
- [35] M. A. Fischler and R. C. Bolles, “Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [36] S. Jiao, Y. Li, and Z. Shan, “Dfs-slam: A visual slam algorithm for deep fusion of semantic information,” *IEEE Robotics and Automation Letters*, 2024.
- [37] Z. Zhang and D. Scaramuzza, “A tutorial on quantitative trajectory evaluation for visual (-inertial) odometry,” in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 7244–7251, IEEE, 2018.